System Design Specification
of
Mobile Application for Real - Time Snake Identification using Convolutional Neural Networks

NEXORA

Name	Registration Number	Index Number
M.A.N.S.G. Samarajeewa	ICT/2021/100	5256
M.E.T. Fernando	ICT/2021/032	5193
M.W. Hansaji Lalithya	ASP/2021/045	5387
M.M.M. Peiris	ICT/2021/087	5243
H.M.R.A. Senevirathna	ICT/2021/110	5265

2025/12/26

ABSTRACT

This System Design Specification documents the detailed design and architectural approach for a Mobile Application for Real-Time Snake Identification using Convolutional Neural Networks (CNN). This system, developed by Group Nexora, is proposed as a vital non-contact method to quickly and safely identify snake species encountered by users.
The system design covers the architecture of the overall solution, detailing the process models, tools, and techniques employed, comprehensive data designs, user interface (UI), user experience (UX) considerations, analytics, and critical security aspects.

The major considerations of this study can be recognized as the CNN model, that will be able to identify the snake and provide details about the identified snake are outlined in this document, with related to the database it consists with.

Furthermore, the SRS addresses the aspects of mobile Application of the system that serves as a core deliverable, making advanced deep learning capabilities accessible to the public and supporting herpetology experts for real-time identification, awareness, and conservation efforts.

ACKNOWLEDGEMENT

The successful completion of this final year project at the Faculty of Applied Sciences, Rajarata University of Sri Lanka, was made possible through the support of many individuals and institutions.
We extend our sincere gratitude to our supervisor, Miss W.B.P.N. Herath, for their continuous guidance, support, and expert mentorship throughout this research.
Our appreciation goes to the academic community and scholars whose research in related fields provided the crucial foundation for our study.We are thankful to all the lecturers and staff of the Information Technology program at the Department of Computing, Faculty of Applied Sciences, Rajarata University of Sri Lanka, for their support and assistance.
Finally, we recognize the hard work, dedication, and cooperation of our project group members in meeting all project deadlines and requirements.
We hope this study contributes positively to the considered area and welcome any interest or future improvements to our work.

Table of Contents

CHAPTER 01: INTRODUCTION 8

1.1 Problem to be addressed 8

1.2 Objectives of the project 8

1.3 Project Deliverables 8

1.4 System design approach 9

1.4.1 The Product Backlog (PB) items 9

1.5 Standards to be followed 12

1.5.1 Coding Standards 12

1.5.2 Documentation Standards 13

1.5.3 Standards for Presentations 13

1.6 Organization of the SDS 13

CHAPTER 02: ARCHITECTURAL DESIGN 15

2.1 The system architecture 15

2.2 Objects and Communication 16

2.2.1 Register 16

2.2.2 Login 17

2.2.3 Snake identification 18

2.2.4 Report Snake Incident 19

2.2.6 Responding to assistance request 20

2.3 Deployment of the system 21

2.4 Processes and special algorithms 21

2.4.1 Forward Propagation Algorithm 22

2.4.2 Error Calculation Algorithm 22

2.4.3 Backward Propagation Algorithm 22

2.4.4 Convolutional Neural Networks (CNN) 22

2.4.5 Geospatial Contextual Filtering 22

2.5 State machines 23

2.5.1 Account Registration 23

2.5.2 Snake Identification 23

2.6 Tools, techniques, libraries, 3rd party tools and implementation environment 24

CHAPTER 03: UI DESIGN 25

3.1 PACT (people, Activities, Contexts, Technologies) analysis 25
3.1.1 People 27
3.1.2 Activities 28
3.1.3 Context 28
3.1.4 Technologies 29
3.2 UI Design consideration and approaches 30
3.3 Design tools, techniques, templates 30
3.4 Input Design aspects 31
3.5 Output Design aspects 33
3.6 Dialogue design aspects 33
3.7 Hosting/ installation environment 33
CHAPTER 04: DATA MANAGEMENT 34
4.1 Data requirements 34
4.2 Design tools, techniques 34
4.3 Conceptual database design 35
4.4 Logical database design 36
3.7.1 Map Regular Entities 36
4.4.2 Map weak Entities 38
4.4.3 Binary Relationships 38
4.4.4 Unary Relationships 41
4.4.5 Map ternary (and n-ary) Relationships 41
4.4.6 Map super type/subtype relationships 42
4.4.7 Map Aggregation 42
4.5 Schema refinement 43
4.6 Physical database design 43
4.7 Security design 44
CHAPTER 05: RESEARCH DESIGN 45
5.1 Study objectives-based literature review 45
5.2 Formalizing high-level implementation components 46
5.3 Data extractions, sample design, test data sets, training data sets 46
5.4 Non-functional aspects 46
5.5 Proposed validation methods and measurements 47
CHAPTER 06: APPROVAL 48

References..... 49

LIST OF TABLES

Table 1 : Functional Requirements of the proposed project 10
Table 2 : Scrum model for the proposed project- Implementation 01 11
Table 3 : SCRUM model for the proposed project - Implementation 02 11
Table 4 : SCRUM model for the proposed project - Implementation 03 12

LIST OF FIGURES

Figure 1 : The System Architecture 15
Figure 2 : Sequence Diagram of Register use case 16
Figure 3: Sequence Diagram of Login use case 17
Figure 4 : Sequence Diagram of Identifying snake use case 18
Figure 5 : Sequence Diagram of Report snake incident use case 19
Figure 6 : Sequence Diagram of Respond to assistance request use case 20
Figure 7 : Component Diagram for the System 21
Figure 8 : Deployment diagram of the system 21
Figure 9 : State Diagram for Account Registration 23
Figure 10 : State Diagram for Snake Identification 23
Figure 11 : User registration Screen 32
Figure 12 : Login Screen 32
Figure 13 : Image acquisition Screen 32
Figure 14 : Profile view Screen 33
Figure 15 : General public and Image realationship 39
Figure 16 : General public and request relationship 39
Figure 17 : General Public and Incident relationship 40
Figure 18 : Snake enthusiast and request relationship 41
Figure 19 : Image and snake relationship ..... **Error! Bookmark not defined.**

CHAPTER 01: INTRODUCTION

The difficulty of distinguishing between different snake species often results in dangerous misidentifications. In the Sri Lankan context, there is an urgent need to address this complexity, as it contributes directly to both a high snakebite burden and the unnecessary loss of biodiversity through the killing of snakes.

1.1 Problem to be addressed

Sri Lanka is lack of an effective, real time and accessible method for snake identification. The comparative analysis of existing research reveals several critical gaps in snake species identification systems. Most studies suffer from limited dataset diversity, often focusing on a narrow range of regional species, which restricts model generalizability. The data quality is frequently compromised due to reliance on online images or controlled environments, lacking real-world context. So, a method that can be used to identify the snakes correctly in an event of a snake encounter will be a valuable support for the people, as well as for the snake enthusiast that is interested in the area of herpetology.

1.2 Objectives of the project

In our study, we propose a real time identification method of snakes in an event of an encounter, to address the problem above. The objectives of the proposed study are,

- To prepare a dataset containing images of different snakes.
- To implement and optimize a preprocessing algorithm that considers camera distortions and light conditions of input image.
- To integrate and adapt a segmentation method to extract the snake and the background separately.
- To develop the deep learning model for identification of snakes.
- To develop a tool to visualize the result processed by the model to the end user.
- Creating a database to keep track of snake images and information.

1.3 Project Deliverables

Project deliverables mentioned below are the outcomes that are expected to achieve as the results of this study.

1. The snake segmentation method.
2. The deep learning model for identification.
3. The Mobile application for real time identification of snakes.
4. The technical reports.

1.4 System design approach

Agile methodology prioritizes iterative development and collaborative problem-solving to deliver functional increments. SCRUM is a popular framework for implementing the agile methodology in a project.

In this study, the SCRUM framework serves as the foundational system design methodology. SCRUM, was chosen for its ability to manage complex workflows through transparency and time-boxed iterations. The following sections detail the application of these principles across three key implementation phases.

In Implementation 01, our focus will be on addressing 60% of the functional requirements as addressed on Software Requirements Specification (SRS) document, and developing a working prototype that includes research components. Following this, Implementation 02 will involve completing the remaining 40% of functional requirements and conducting unit testing. Finally, in Implementation 03, we will concentrate on integration and integration testing, acceptance testing, and evaluating the research components.

1.4.1 The Product Backlog (PB) items

The Product Backlog serves as the single source of truth for all work required to achieve the project objectives. It is comprehensive, encompassing not only functional requirements (User Stories and Features) but also architectural and non-functional work, such as database design, system integration, and dedicated quality assurance/testing activities.

Functional Requirements of the project

No	Functional Requirements
1	Submit Registration
2	Validate Registration form details
3	Check for Email existence
4	Create user account
5	System sends account activation email.
6	DB Driver load
7	DB Connection creation
8	Data retrieval from the DB
9	Update Account activation status
10	Manipulation of the Data of DB
11	Submit Login
12	Validate Login form details
13	Verify uploaded/ captured image
14	Store image in the Database

15	Snake segmentation for uploaded image by user
16	Snake identification from the uploaded image by user
17	Password forgot form data submits
18	Forgot password validation
19	Modify own account information
20	Own account modification validation

Table 1 : Functional Requirements of the proposed project

Product Backlog Items

No	PB Item	Story point
1	Functional Requirements – 01 Implementation	04
2	Functional Requirements – 02 Implementation	04
3	Database Creation	05
4	Preprocessing Algorithm	05
5	Segmentation Method	05
6	Identification Model	03
7	Unit Testing	05
8	Experimental set up of Preprocessing Algorithm	03
9	Experimental setup of Segmentation Method	03
10	Experimental setup of Identification Model	03
11	Integrating the model with mobile application	05
12	Integration testing	05
13	Evaluate research components of the application	04
14	Acceptance Testing	04

Table 2 : Product Backlog Items

Implementation 01

- Total points = 23 points (04+04+05+05+05)
- Estimated Velocity (number of story points a team actually completes in a sprint) = 05 points
- Time sprints required = 23 /5 ≈ 05 sprints

Implementation 02

- Total Points = 17 points (03+05+03+03+03)

- Estimated Velocity = 05 points
- Time sprints required = 14/05 ≈ 03 sprints

Implementation 03

- Total Points = 18 points (05+05+04+04)
- Estimated Velocity = 05 points
- Time sprints required = 18/05 ≈ 04 sprints

	Product Backlog Items (PB Items)	**Sprint 01 (06 Days)**	**Sprint 02 (06 Days)**	**Sprint 03 (06 Days)**	**Sprint 04 (06 Days)**	**Sprint 05 (06 Days)**
Implementation 01 (30 days)	60% of Functional Requirements	30%	30%	20%	10%	10%
	Database Creation	30%	30%	20%	10%	10%
	Preprocessing Algorithm	20%	20%	30%	20%	10%
	Segmentation Method	20%	20%	20%	20%	20%
	Identification Model	10%	10%	10%	30%	40%

Table 3 : SCRUM model for the proposed project- Implementation 01

	Product Backlog Items (PB Items)	**Sprint 01 (10 Days)**	**Sprint 02 (10 Days)**	**Sprint 03 (10 Days)**
Implementation 02 (30 days)	40% of Functional Requirements	50%	50%	
	Unit Testing	40%	30%	30%
	Experimental setup of Preprocessing Algorithm	40%	60%	
	Experimental setup of Segmentation Method	30%	70%	
	Experimental setup of Identification Model		50%	50%

Table 4 : SCRUM model for the proposed project - Implementation 02

	Product Backlog Items (PB Items)	Sprint 01 (7.5 Days)	Sprint 02 (7.5 Days)	Sprint 03 (7.5 Days)	Sprint 04 (7.5 Days)
Implementation 03 (30 days)	Integrate the model with mobile application	50%	50%		
	Integration testing	50%	50%		
	Evaluate research components of the application			100%	
	Acceptance Testing				100%

Table 5 : SCRUM model for the proposed project - Implementation 03

1.5 Standards to be followed

We have adopted a structured set of standards to streamline the research process and effectively meet our objectives

Here the study is expected to follow standards in three different aspects.

1.5.1 Coding Standards

a. PEP 8

PEP 8 (Python Enhancement Proposal 8) is the official style guide for Python code. It provides a set of widely accepted conventions and best practices for how to format Python code to ensure consistency, improve readability, and make it easier for developers to collaborate on projects.

b. Modular Design

Implementing a modular design strategy involves architecting source code into discrete, self-contained components that each serve a dedicated, well-defined purpose. By decoupling complex systems into these manageable units, developers establish a clear separation of concerns, ensuring that individual functionalities remain isolated and reducing the risk of unintended side effects when making updates.

c. Error Handling

Implement informative error handling by providing clear user-facing messages and comprehensive logs for backend diagnostics

d. Security

Follow security best practices to protect sensitive data and prevent Vulnerabilities.

e. Version Control

Track all development in Git, following best practices for commits and merges. Use pull requests and code reviews as primary tools for effective team collaboration and quality assurance

1.5.2 Documentation Standards

a. IEEE standard

All study documentation adheres to IEEE standards and guidelines. This ensures consistency across citations, references, figures, tables, and mathematical formulas, while maintaining standardized usage of abbreviations and acronyms.

1.5.3 Standards for Presentations

a. Clarity and Conciseness

Clearly define the primary purpose and specific objectives of the presentation upfront to ensure focus and coherence. Maintain high-level conciseness by limiting each slide to one central idea or topic, rigorously avoiding excessive text or dense information.

b. Consistent Design

Maintain a unified design theme throughout the entire deck. This includes adhering to defined corporate standards for fonts, color palettes (e.g., primary and secondary colors), and layout schemas to reinforce organizational identity and professionalism.

c. Practice and Rehearsal

Thoroughly practice delivering the presentation multiple times to ensure complete fluency, projected confidence, and smooth, timely transitions between slides.

1.6 Organization of the SDS

The rest of the SDS Document will be organized as follows;

1. Architectural Design

   The system's architecture, including its objects, communications, procedures and specialized algorithms, is described in the architectural design section. This chapter's next section will give a quick summary of state machines, tools, methods, third-party tools, libraries, and the environment in which they are implemented.

2. UI Design

   The PACT (People, Activities, Contexts, and Technologies) study will be described in the third chapter, and then UI design concerns and methods will be examined. In the middle of the chapter, we'll explore design methods, templates, and tools. We will also talk about dialogue

design, input, and output. Lastly, a review of the hosting and installation environment will round up the chapter.

3. Data Management

Data requirements will be discussed at the beginning of the fourth chapter. The design tools and methods used during the project will next be examined. We will then explore logical, conceptual, and physical database designs. A review of database security measures and schema refinement will round up the chapter.

4. Research Design

We start the fifth chapter by formalizing high-level implementation components and looking at the goals that came from our literature review. As the chapter goes on, we concentrate on data extraction, sample design, and the generation of training and testing data sets. We discuss non-functional aspects in the middle of the chapter and offer validation techniques and metrics at the end.

CHAPTER 02: ARCHITECTURAL DESIGN

2.1 The system architecture

The system architecture is comprised of three primary components, a mobile application, an administrative web interface, and a cloud server. Mobile Application is divided into the User Interface (UI) and the backend. The UI, along with the web interface, facilitates visual presentation and user interaction, while the backend manages the core functionality and application logic.

Cloud server hosts the web framework, a centralized database, and two specialized models. The database serves as a distinct repository within the server environment. Two functional models perform specific, differentiated tasks essential to the execution of the inpainting process.

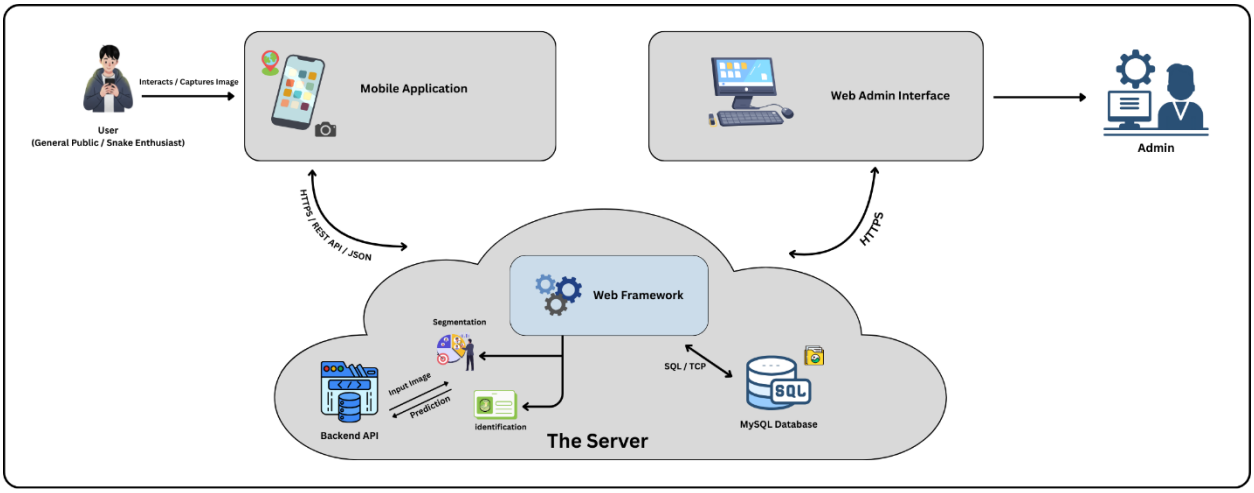


Figure 1 : The System Architecture

2.2 Objects and Communication

2.2.1 Register

The following Figure 2 illustrates the sequence of interactions between the classes involved in the registration process. The process begins when the user clicks the Signup Button, prompting the interface to load the registration form. Once the user completes and submits the form, the Registration Form Handler validates the input. If the inputs are invalid, the user is prompted to re-fill the form, otherwise, the data is passed to the Registration Handler. The Registration Handler queries the database to determine if the provided email already exists. If the email is already registered, the system issues a warning to the user. If the email is unique, the handler creates a new user account and triggers the Mail Handler to send a registration activation email.

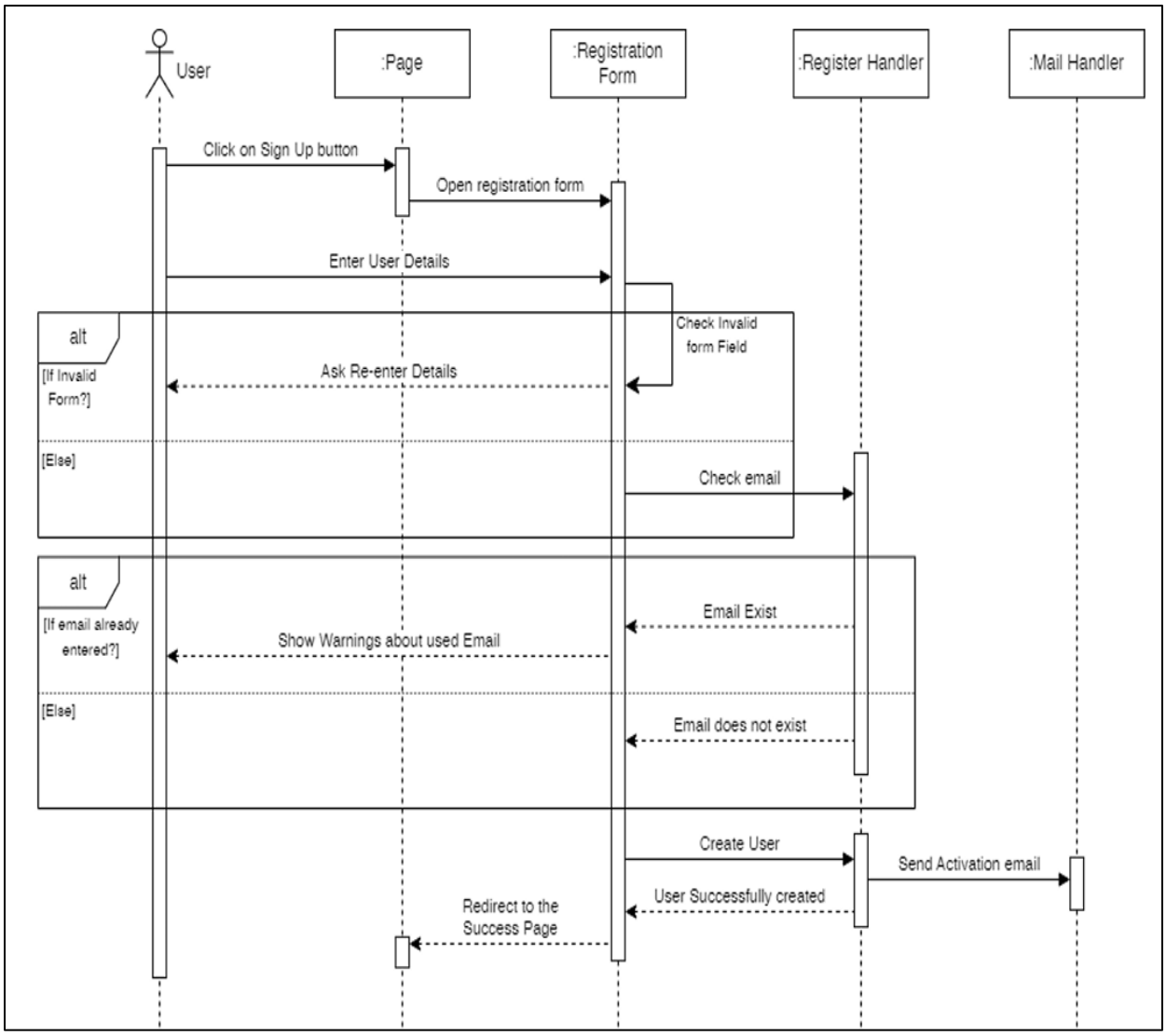


Figure 2 : Sequence Diagram of Register use case

2.2.2 Login

Figure 03 below illustrates the authentication process for registered users. The User initiates the process by entering their credentials on the Login Screen. The application first validates the input format (e.g., ensuring fields are not empty) before sending a secure POST /login request to the Backend API. The system then retrieves the user's account details and password hash from the Database to verify the credentials. If the password verification fails or the account does not exist, the system returns an error message, and the user is prompted to retry.If the credentials are valid, the Backend generates a session token and returns a success response. The user is then redirected to the appropriate Home Screen based on their user role.

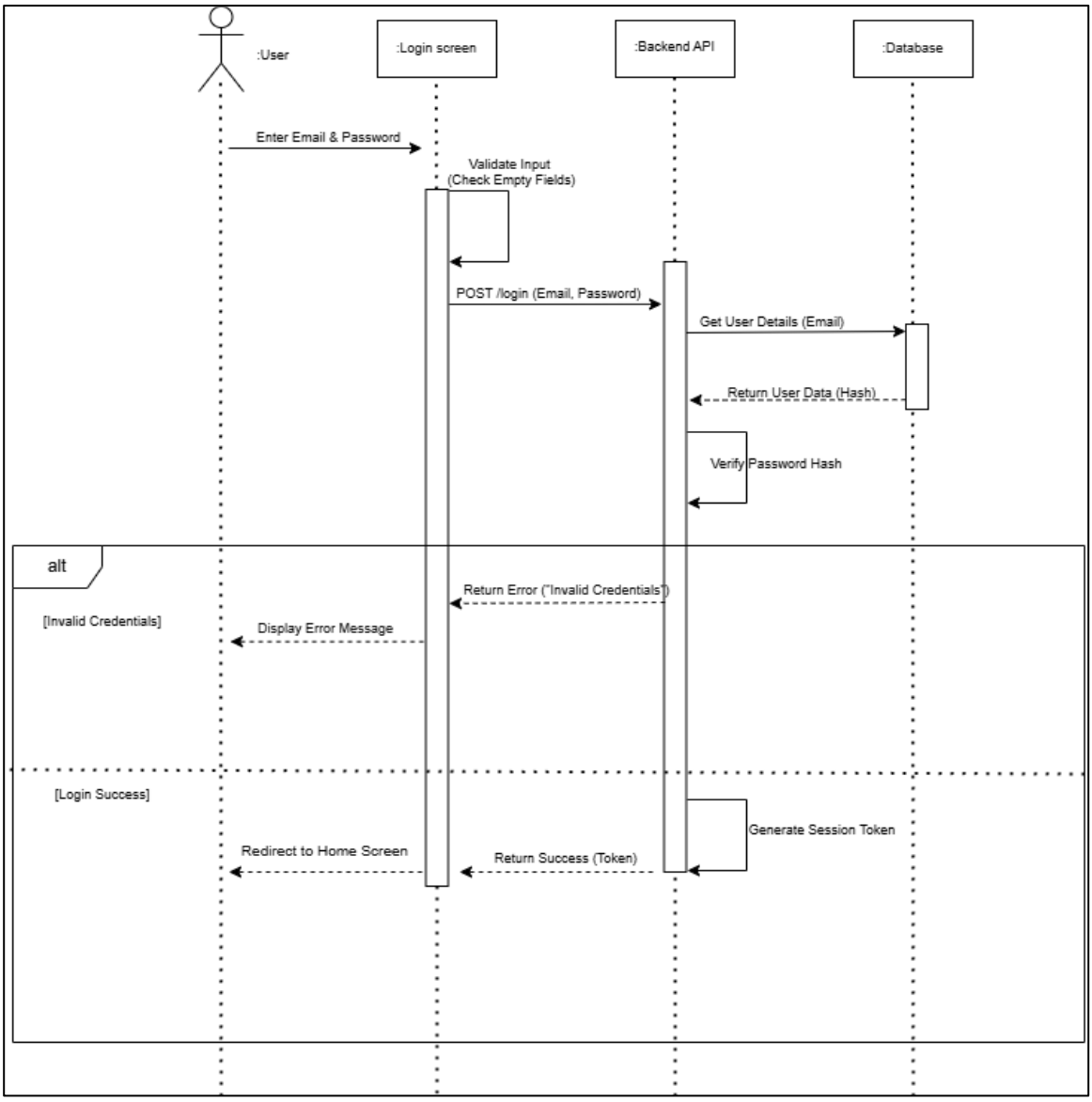


Figure 3: Sequence Diagram of Login use case

2.2.3 Snake identification

This diagram depicts the real-time snake identification workflow. The User captures or uploads an image via the Mobile App, which is sent along with the user's current GPS location to the Backend API. The system first records the incident in the Database for historical tracking. The image then undergoes pre-processing (resizing and normalization) before being analyzed by the Convolutional Neural Network (CNN) Model.If the model identifies the species with a high confidence score, the system retrieves the specific species details (Venom status, First Aid) from the Database and displays them to the user. If the model cannot identify the snake with sufficient certainty, a warning message is displayed, advising the user to exercise caution.

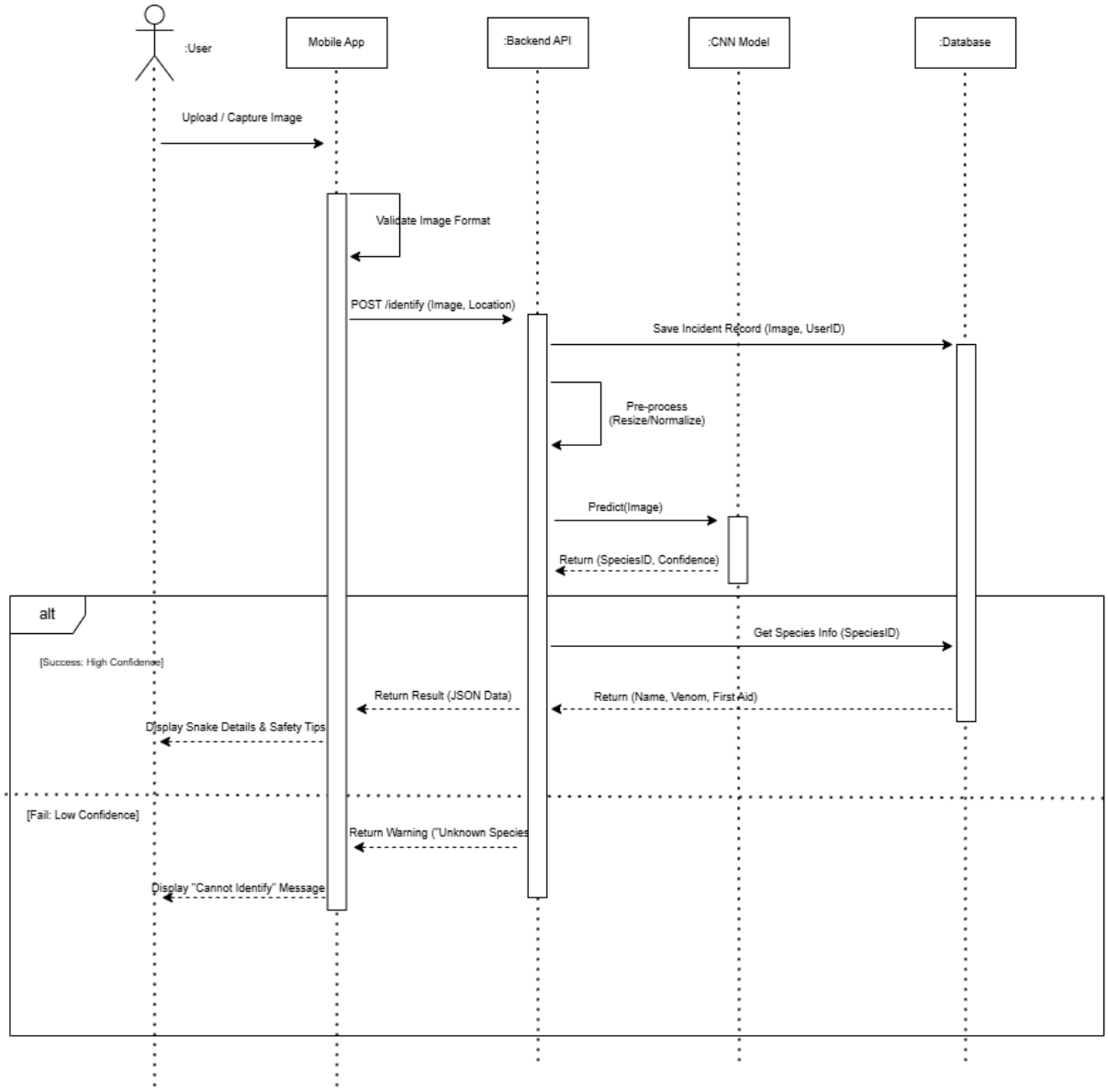


Figure 4 : Sequence Diagram of Identifying snake use case

2.2.4 Report Snake Incident

This diagram illustrates the workflow for reporting a snake sighting. The process begins when the User initiates a report. The Mobile App automatically requests GPS coordinates from the location service to geotag the incident.The User then submits the incident details (image, description, and assistance preference). The Backend API validates the data and stores the incident record in the Database.If the user checks the "Request Assistance" option, the system queries the Database for Snake Enthusiasts within a set radius. The Backend then triggers the Notification Service to send a push alert to the identified enthusiasts.If assistance is not requested, the system bypasses the notification step and returns a success confirmation to the user.

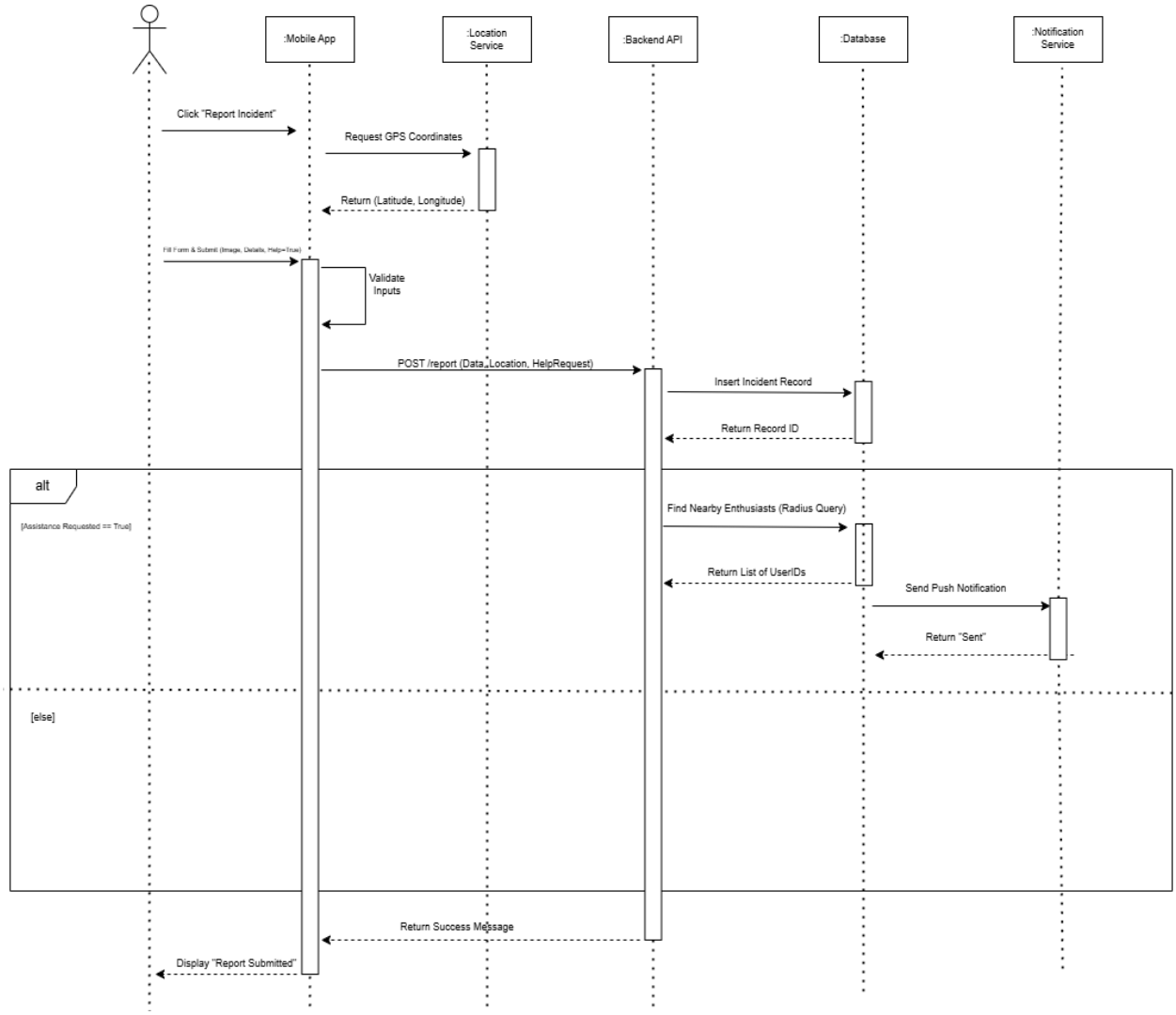


Figure 5 : Sequence Diagram of Report snake incident use case

2.2.6 Responding to assistance request

This diagram illustrates the workflow when a Snake Enthusiast responds to an emergency assistance request. The process begins when the Enthusiast receives a push notification regarding a nearby snake incident. Upon tapping the notification, the Mobile App retrieves the full incident details (location, images, and user notes) from the Backend API.The Enthusiast then decides whether to accept or decline the request:If the Enthusiast accepts the request, the system updates the incident status to "Assistance Accepted" in the Database. Crucially, the Backend immediately triggers the Notification Service to send a push alert to the original User, informing them that help is on the way.If the Enthusiast declines, the system simply logs the response in the Database and takes no further action towards the original user, allowing other enthusiasts to respond.

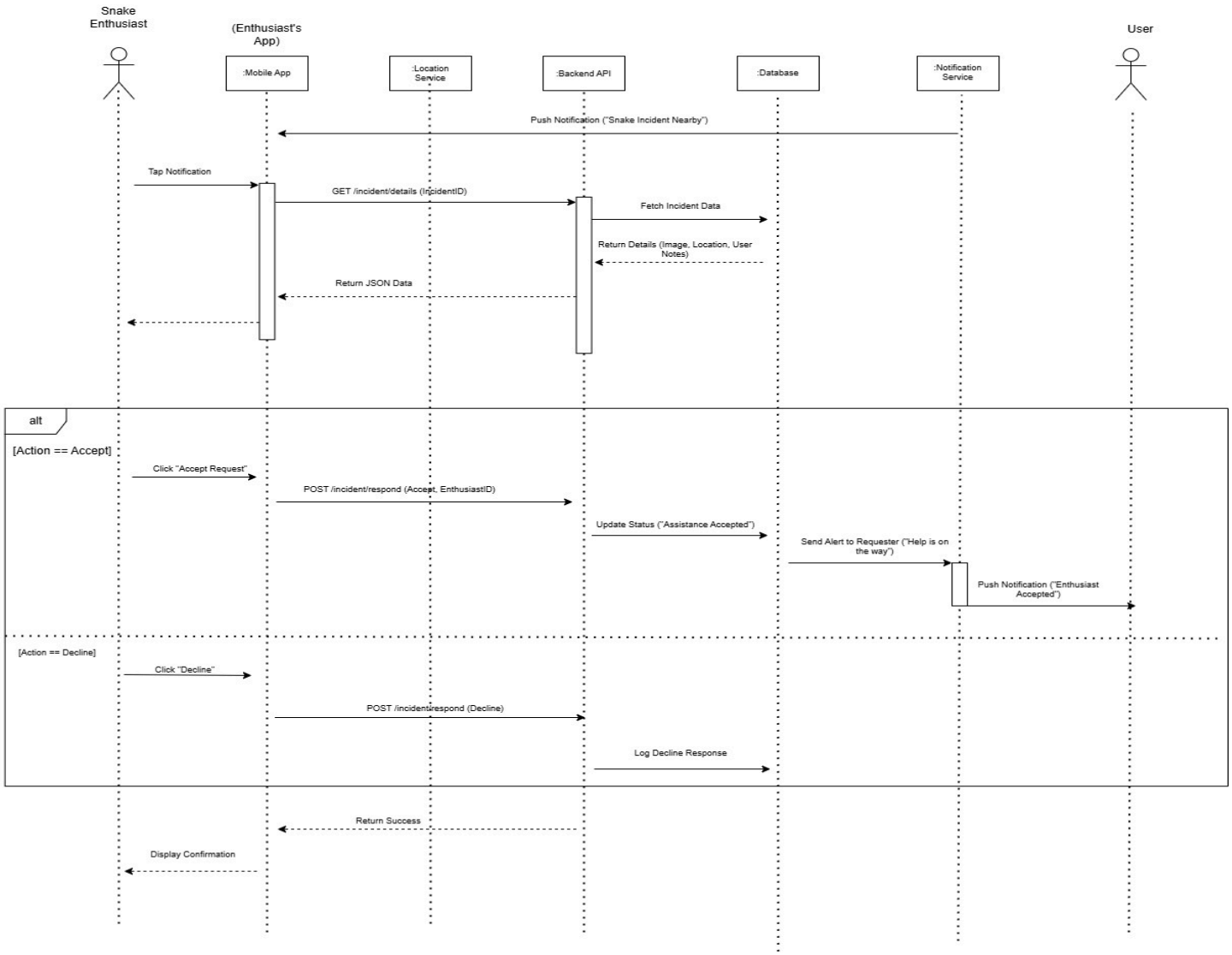


Figure 6 : Sequence Diagram of Respond to assistance request use case

2.3 Deployment of the system

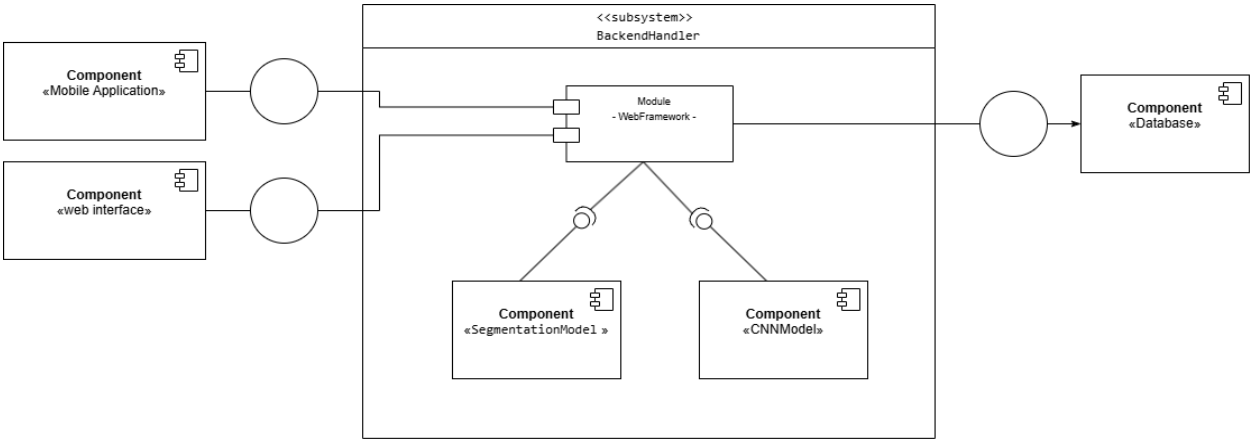


Figure 7 : Component Diagram for the System

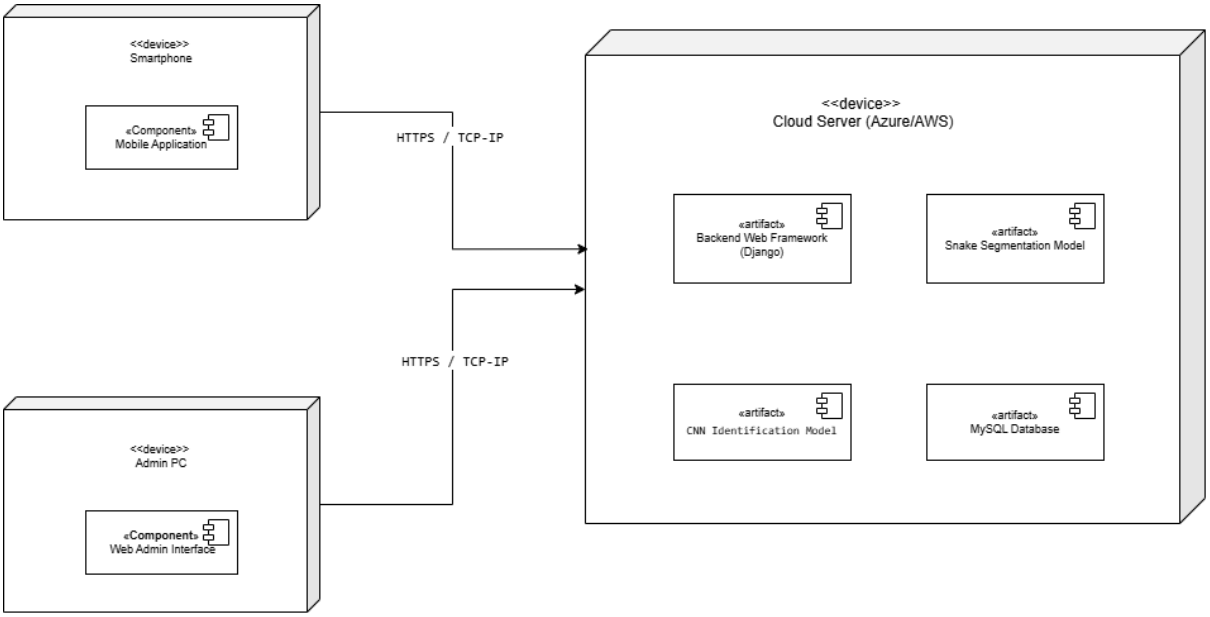


Figure 8 : Deployment diagram of the system

2.4 Processes and special algorithms

In Processes and special algorithms include the algorithms that are going to be used for the system device. And, the processes which we are using in the system. There are several algorithms and processes related to our system. In this section we describe the main processes and the main algorithms that are important to our system.

2.4.1 Forward Propagation Algorithm

Forward propagation is the process where input data travels through the neural network's architecture, moving sequentially from the input layer toward the output layer. During this phase, neurons in every layer compute weighted sum of their inputs, execute specific activation functions, and transmit the resulting values to the subsequent layer. This continuous flow culminates in the production of a final output, which serves as the network's prediction for the provided data.

2.4.2 Error Calculation Algorithm

This stage involves using a mathematical loss function to determine the numerical difference between the network's predicted result and the actual target value, known as the ground truth. Common metrics used for this measurement include Cross-Entropy Loss or Mean Squared Error (MSE). The resulting error value acts as a performance indicator, providing the necessary data to inform and drive the model's optimization process.

2.4.3 Backward Propagation Algorithm

Backpropagation identifies how much each weight and bias contributes to the total error by calculating gradients in reverse, starting from the output and moving toward the input layer. By applying the mathematical chain rule, the system determines the specific impact of every parameter on the overall discrepancy. These calculated gradients are then utilized by optimization techniques, such as gradient descent, to refine the weights and biases. The primary objective of this iterative adjustment is to minimize the loss and enhance the model's predictive precision.

2.4.4 Convolutional Neural Networks (CNN)

CNNs are specialized architectures consisting of several functional layers designed for feature extraction and classification:

- Convolutional Layer**:** Utilizes a series of learnable filters to detect and extract significant patterns or high-level features from the input.
- Activation Layer: Implements non-linear functions, such as Rectified Linear Unit ($ReLU$), to allow the model to learn complex patterns.
- Pooling Layer: Downsamples the data to decrease spatial dimensions, which lowers computational requirements while preserving essential information.
- Fully Connected Layer: Acts as the final stage of the network to produce the ultimate regression or classification results.

2.4.5 Geospatial Contextual Filtering

To improve accuracy, the system employs a contextual filtering algorithm using GPS data.

1. Location Capture**:** The app retrieves the user's current latitude and longitude.

2. Contextual Matching: The predicted species is cross-referenced with the known geographical distribution of that snake in Sri Lanka.

If a predicted species is not native to the user's current location, the system may flag the result for lower confidence or suggest alternative species common to that region.

2.5 State machines

State machines are behavioral diagrams used to model the dynamic lifecycle of a single object or a system. While a flowchart describes a sequence of activities, a state diagram focuses on the states an object occupies and how it moves between them in response to specific events.

2.5.1 Account Registration

After the user submits the registration form, the account status is initialized as 0, indicating a pending or inactive state. However, when the user interacts with the activation link, the account status transitions to 1, indicating successful activation and granting access to the account's functionalities.

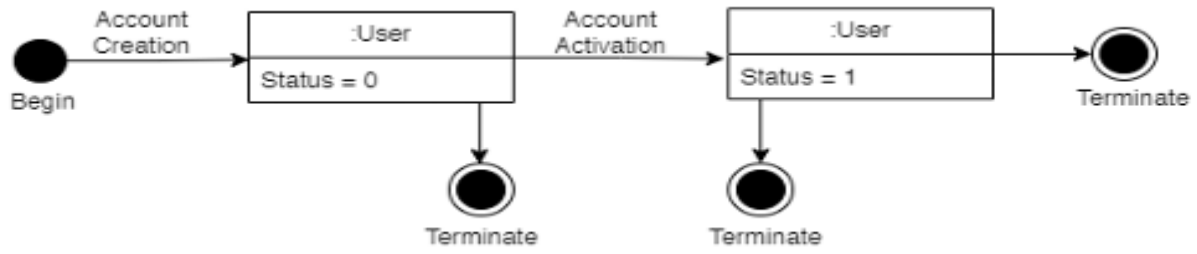


Figure 9 : State Diagram for Account Registration

2.5.2 Snake Identification

Snake Identification system first process the image in image segmentation model.If segmentation success only the image pass into identification module.Otherwise,system reject the uploaded image and ask for re upload another image.

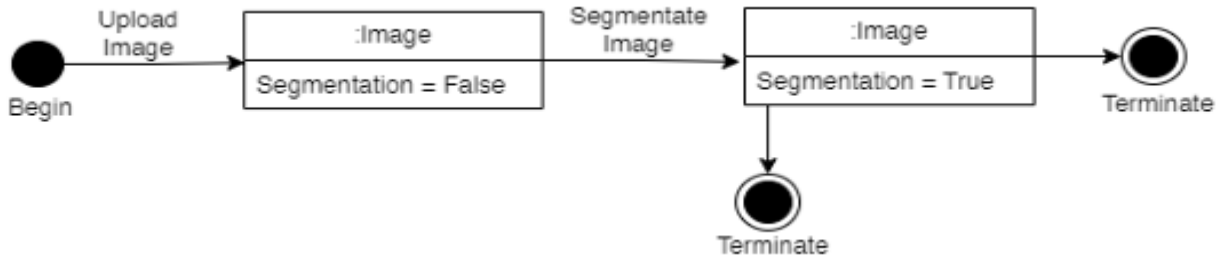


Figure 10 : State Diagram for Snake Identification

2.6 Tools, techniques, libraries, 3rd party tools and implementation environment

Category	Tool / Technology	Description
Mobile Development	Android Studio	The primary Integrated Development Environment (IDE) used for building the Android mobile application.
Programming Languages	Java / Python	Java is used for Android app development, while Python is used for backend logic and model training.
Machine Learning	TensorFlow / Keras	An open-source library used to design, train, and deploy the CNN model for snake image classification.
Cloud Environment	Google Colab	A cloud-based platform used for training the deep learning model, utilizing GPU acceleration.
Database Management	MySQL	A relational database management system used to store user data, snake species records, and incident reports.
UI/UX Design	Figma	A collaborative interface design tool used for creating wireframes and prototyping the mobile application screens[20].
Diagramming	Draw.io	Used for creating UML diagrams such as Use Case, Sequence, and Activity diagrams for the system design[21].

CHAPTER 03: UI DESIGN

A crucial part of software design is user interface (UI) design, which concentrates on the application's interactive and visual elements. It entails creating the application's user interface while paying close attention to layout, typography, color palettes, and graphics components such as pictures and icons. The purpose of this part is to describe the UI design guidelines that will direct the software's functional and aesthetic elements to guarantee that it is both aesthetically pleasing and easy to use.

To guarantee that the program is user-friendly for all users, the UI design process will follow best practices in usability, accessibility, and user experience (UX). Wireframing, prototyping, and iterative testing are some of the phases that will be used to produce these specifications. These acts are necessary to improve user interactions and maximize the software's overall usability.

In addition to increasing customer happiness, a well-designed user interface (UI) encourages greater engagement and software involvement. The UI design will facilitate successful and efficient usage of the software if the rules outlined in this specification are followed.

3.1 PACT (people, Activities, Contexts, Technologies) analysis

In human-computer interaction (HCI), PACT is a scoping framework used to analyze the situational context of a system. It ensures that designers don't just focus on the software, but also on the humans using it, what they are doing, and the environment they are in. The abbreviation PACT stands for People,Activities,Context and Technologies – each representing critical dimensions to be analyzed for informed decision- making in the design process.

Screen	People	Activities	Context	Technology
Login Screen	Multilingual users with mixed digital literacy → Simple language, Sinhala/Tamil/English toggle placed at the top	First-time users must complete registration quickly → Step-by-step guided form with inline validation.	Users may register in outdoor / low-attention environments → Large touch targets, high-contrast labels, minimal scrolling.	Touch-optimized text fields, numeric keyboards for phone number, secure password masking.
Registration Screen				
Account Activation Screen	Users who have submitted registration and are waiting for verification	Interacting with an activation link to transition account status		Mail Handler integration to trigger and verify

		from "pending" to "active".		unique activation tokens.
Verify Password Screen	Registered users who have forgotten their credentials and need to prove identity.	Submitting a "forgot password" form and validating identity via a secure code.	High-stress or urgent context if trying to log in during a field encounter.	Secure OTP (One-Time Password) or link-based validation protocols
New Password Screen				
Account Deactivation Screen				
Home Screen	All user types (General Public, Enthusiasts, Admins) needing quick access to core tools.	Transitioning between snake identification, reporting, and viewing collections.	Highly variable; used in transit, rural areas, or bright sunlight	Adaptive UI with a bottom navigation bar or swipeable pages for fluid transitions
Account modification Screen				
Upload/Import Image Screen	Users may be anxious or inexperienced → On-screen instructions in simple language with visual hints.	Core task is image capture → Single clear capture button, auto-focus and blur warnings.	Used outdoors with bright sunlight → High-contrast UI, minimal text, clear white background.	Live camera API, auto-exposure, image quality validation before upload.
View snake details Screen				
View collection Screen				
Report snake incident Screen	Civic-minded users wanting to help community → Positive reinforcement UI ("Thank you for reporting").	Geo-tagging and describing incident → Auto-fill GPS and Time to reduce typing effort.	Time-sensitive context, potentially unstable signal → Offline-first design that queues reports.	Google Maps API, Background Sync Services, Text-to-Speech input.
Request snake enthusiast Screen	Users in potential danger needing expert help → Clear	Sending SOS alert → Single-tap "Request	High stress / Emergency context →	Geolocation API, WebSocket for

	distinction between "Emergency" and "Inquiry".	Help" button with location sharing.	Minimal steps, direct visual confirmation of help.	real-time status updates.
View own account Screen				
Response to requests Screen	Enthusiasts responding to a request notification → Needs accurate location data.		Mobile / Transit context → Hands-free navigation support.	Map Routing API, Real-time chat/call integration.
Admin Login Screen				
Admin Operation Screen				

3.1.1 People

A core pillar of the PACT framework involves a comprehensive characterization of the User dimension to inform the software's architectural and interface requirements. This analysis systematically evaluates user demographics, cognitive load under stress, technical proficiency, and specific functional goals. By adopting a user-centered design (UCD) philosophy, the system architecture is engineered to mitigate operational complexity and maximize intuitive interaction. These behavioral insights directly dictate the interface's navigational hierarchy and functional priorities, ensuring the final product delivers high utility and a seamless user experience in critical real-world scenarios.

- **Language (English/Sinhala/Tamil)**

Our application is localized in English, Tamil, and Sinhala, the three primary languages of our users. We are committed to using clear, concise, and straightforward language across all translations to ensure the content remains easily accessible to both native and non-native speakers. By avoiding complex jargon and prioritizing readability, we aim to provide a seamless and inclusive user experience regardless of the user's primary language.

- **Interaction Level**

The app is meant to be easy to use for everyone, from experts in herpetology to general public who uses the app to identify snakes. The interface should be straightforward but also allow for deeper engagement for those who desire it.

- **Usage**

The app is engineered for maximum versatility, functioning as a reliable companion for both intensive academic research and active, on-site learning in the field. To support this range of use cases, the design utilizes an adaptive interface that adjusts to the user's environment, ensuring optimal performance and accessibility. By prioritizing this contextual flexibility, the app empowers users to transition seamlessly between diverse learning environments without sacrificing efficiency or focus.

3.1.2 Activities

To optimize the system for real-time performance, we have analyzed the core user activities through a functional task-analysis lens. Our design prioritizes the critical path of the snake identification process: from initial camera engagement and frame capture to data validation and the management of system feedback messages. By scrutinizing the sequence and frequency of these actions—particularly regarding legibility under duress and rapid data selection—we have engineered an interface that reduces cognitive load. This alignment between design and activity ensures that the user can navigate the identification workflow with maximum precision and minimum effort, even in high-pressure environmental contexts.

- **Data Entry**

When using the app for the first time, users are prompted to enter information such as their name and email address details. This process should be straightforward, guiding users through each step for a smooth experience.

- **Selecting Options**

Users can choose from various analysis types or specific statues for detailed examination. This selection process should be intuitive, enabling easy navigation through options.

- **Navigation**

It is crucial that users can fluidly transition between different sections of the app, accessing features effortlessly and without confusion.

- **Camera Use**

This function allows users to capture images of headless statues for analysis. Integrating this capability with clear, simple instructions will improve the overall user experience.

3.1.3 Context

The context in which our software is used plays a pivotal role in shaping the design and implementation of the technology. This includes the physical environment, social dynamics, cultural settings, and organizational structures. Understanding these contexts is crucial to ensure that our software is adaptable and effective across different settings. It influences everything from

user interface design to functionality and integration capabilities. By carefully considering these various contexts, we can tailor our software to better meet the specific needs and constraints of our users in their unique environments.

- **Personal Use**
  For individuals who engage in hiking ,wild life exploring and farming ,the app should be engaging and useful,providing detailed information in a straightforward manner.It should work well in various personal settings,from bright outdoor locations to more subdued indoor environments.

- **Educational purposes**
  The application can be a valuable tool for educational puposes,providing a detailed information about different snake species.

- **Wild and rural areas**
  These environments are primary places where the app is likely to be used.The app can help people to identify the snake in an event of a incident.The interface should be easy to use and function well in conditions like low light and low connection.

3.1.4 Technologies

Technologies play a critical role in optimizing the user interface (UI) for the specific devices and platforms our software will operate on. It is vital to select and utilize the appropriate technologies that enhance UI responsiveness, compatibility, and performance across all user environments. This consideration ensures that our software delivers a consistent and effective user experience, regardless of the device or platform used. By strategically leveraging these technologies, we can ensure our software meets modern standards and user expectations.

- **Input Methods**
  The app should offer easy-to-use input methods tailored to touch devices.

- **Navigation**
  Navigation within the app should be straightforward, allowing users to move between different sections with ease. Consistent and intuitive navigation aids, like bottom navigation bars or swipe able pages, help ensure that users can find what they need without frustration.

- **Output Methods**
  The app needs to display information and visual content in a way that is easy to understand. This includes the use of high-quality images and animations that can enhance the user's learning experience, ensuring that outputs such as identifying snakes are clear and precise.

3.2 UI Design consideration and approaches

When designing the user interface (UI) for the mobile app, several key considerations and approaches are pivotal to enhancing the user experience. This section outlines these essential UI design strategies to ensure that our application is not only functional but also intuitive and engaging for users. We focus on optimizing usability, aesthetics, and interaction to meet the specific needs of this unique application.

• User-Centric Design
This approach ensures the interface is designed based on what users need and prefer. By researching and collecting feedback from users, the design aims to be easy to use and focused on solving user problems.

• Clear and Consistent Layout
Important for making sure the interface is easy to understand. It involves using a uniform style and arrangement, where similar items are grouped together. This helps users find information quickly and easily.

• Intuitive Navigation
Allows users to smoothly navigate through the app with familiar tools like tabs and menus that are easy to find and use. This helps users get to what they need without confusion.

• Reliability and Legibility
Focuses on making the interface reliable and easy to read. Text should be clear and fonts should be simple, making sure users can read easily in different conditions.

• Visual Design and Approach
Involves choosing colors, styles, and images that make the interface look good and engage users. This is about making the app not just functional but also visually appealing.

3.3 Design tools, techniques, templates

In the realm of UI design for our software, choosing the appropriate tools, techniques, and templates is essential for crafting effective and visually appealing interfaces. This section explores the specific tools and methodologies we employ to streamline our design process and ensure consistency and quality across our user interfaces.

3.3.1 Design Tools

- **Figma**

Figma is a popular tool among UI designers for its cloud-based collaboration capabilities and cross-platform compatibility. It offers powerful design and prototyping features, supports a component-based design approach, and provides robust support for design systems. Additionally, Figma facilitates developer handoff with its integration options. Its user-friendly interface and responsiveness to user feedback help maintain its standing as a top choice for design tools.

- **Adobe Photoshop**

Adobe Photoshop is widely recognized for its advanced features in editing and manipulating digital images. It offers a variety of tools for tasks such as photo retouching, adjusting colors and lighting, as well as cropping and resizing images. With capabilities including layers, masks, filters, and selection tools, Photoshop is used for both simple photo enhancements and complex digital artwork by photographers, digital artists, and graphic designers.

3.3.2 Design Techniques

- **Prototyping**

Prototyping involves building interactive versions of the UI that allow for the simulation of user interactions. This is a critical step for testing design concepts and gathering feedback from users or stakeholders to refine the interface before final implementation.

- **Information Architecture**

This aspect focuses on organizing the content and navigation of the platform to ensure that users can access information smoothly and logically. Effective information architecture supports intuitive navigation and enhances user experience by structuring content in a clear and logical manner.

3.4 Input Design aspects

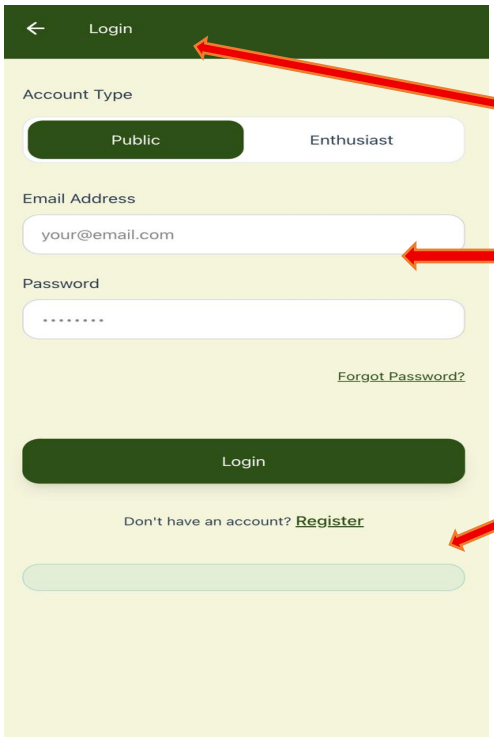


Figure 12 : Login Screen

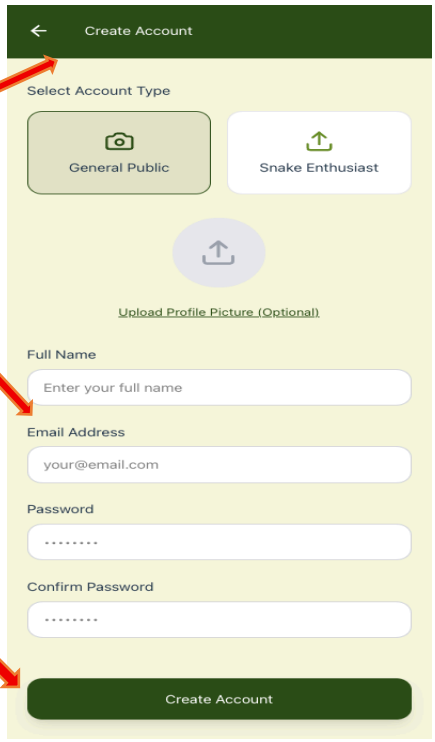
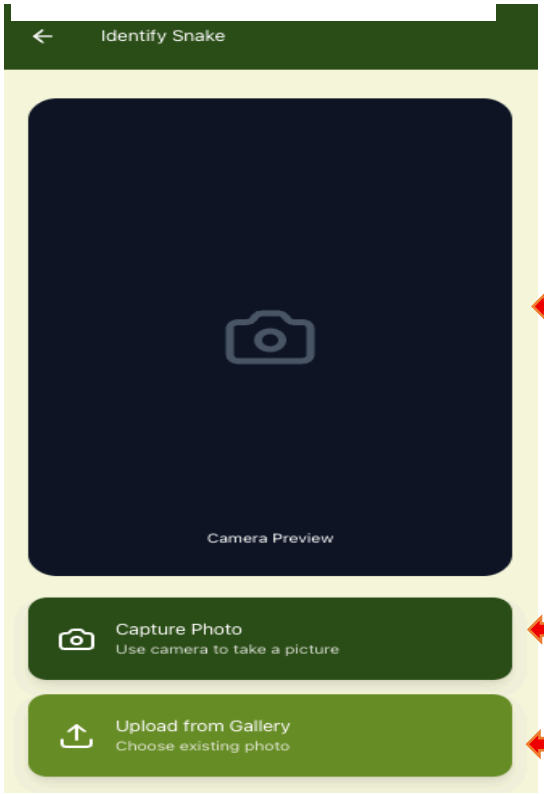


Figure 11 : User registration Screen

Title of the page

Input fields

Sign In/ create account Buttons



Camera Display

Capture Button

Gallery upload Button

Figure 13 : Image acquisition Screen

These figures display the UI interfaces of the Input design aspect which are the Login,User registration and Image Acquisition.
For the simplicity of the program,input design aspects are aimed to use minimum required components as much as possible.

3.5 Output Design aspects

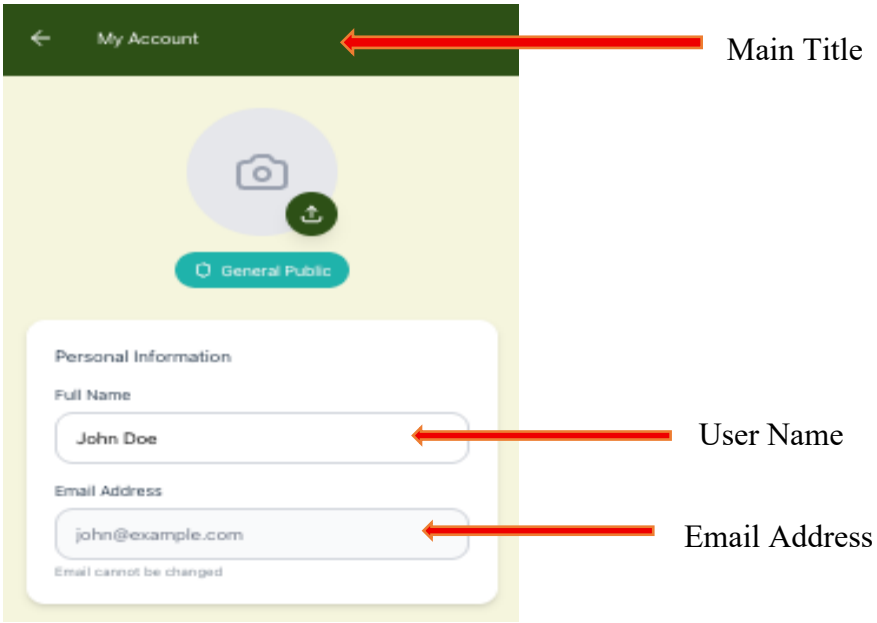


Figure 14 : Profile view Screen

3.6 Dialogue design aspects

3.7 Hosting/ installation environment

The Mobile application is designed to operate within a robust hosting and installation environment tailored to support its advanced features. This environment includes a dedicated server setup capable of handling complex data processing and image analysis. Essential components such as a SQL database for data management and specialized models for identification of snakes that ensure efficient operation and storage solutions. For user installation, the app requires compatibility with Android devices running version 7.0 or above. To deliver an optimal user experience, devices must have at least a 8 Megapixel camera for high-quality image capture, and a minimum of 4GB RAM to handle the app's processing demands effectively. These specifications are critical to achieving precise visualization outputs and maintaining smooth application performance across supported devices.

CHAPTER 04: DATA MANAGEMENT

Data management is a critical pillar of project success, especially when handling sensitive user information. Maintaining database accuracy and robust security is essential for seamless operations and the preservation of privacy. This process encompasses defining data requirements, architecting the database, optimizing organizational structures, and implementing rigorous security protocols. By integrating these core components, organizations ensure both system reliability and protection against emerging threats.

4.1 Data requirements

The primary goal of the data management segment is to gather information about different kinds of snakes and their characteristics and store it in a database.
We will use image processing techniques on the photos we get as inputs and compare them to the information we recorded in the database. Data needs should be properly planned out in order to obtain these outcomes exactly. The project's data needs are listed below.

• Store the details of the snake types
• Store the details of the snake locations
• Store the details of the snake venom type

4.2 Design tools, techniques

- **MYSQL**

  While MySQL is technically a database management system (DBMS) rather than a design tool, it is the fundamental platform where your data architecture comes to life. By choosing MySQL, you are opting for a battle-tested, reliable engine that excels at organizing and storing complex datasets through its structured relational model. Its ability to handle large volumes of information with high performance—facilitated by efficient indexing and rapid retrieval—makes it an ideal backbone for any data-driven project. Furthermore, its broad compatibility with various programming languages ensures seamless system integration, while its sophisticated security protocols provide a robust defense for sensitive user data. Ultimately, MySQL's balance of efficiency, flexibility, and proven security makes it an essential choice for maintaining project integrity and long-term scalability.

- **Figma**

  Figma bridges the gap between raw data and the end-user by focusing on how information is presented and manipulated within the application's interface. It allows designers to prototype the "front-facing" side of data management—such as input forms, data validation feedback, and dashboard visualizations—ensuring that complex data remains accessible and intuitive. Through plugins that pull in real JSON or spreadsheet data, Figma helps teams verify that the database structure effectively supports the user's needs and the app's visual layout.

4.3 Conceptual database design

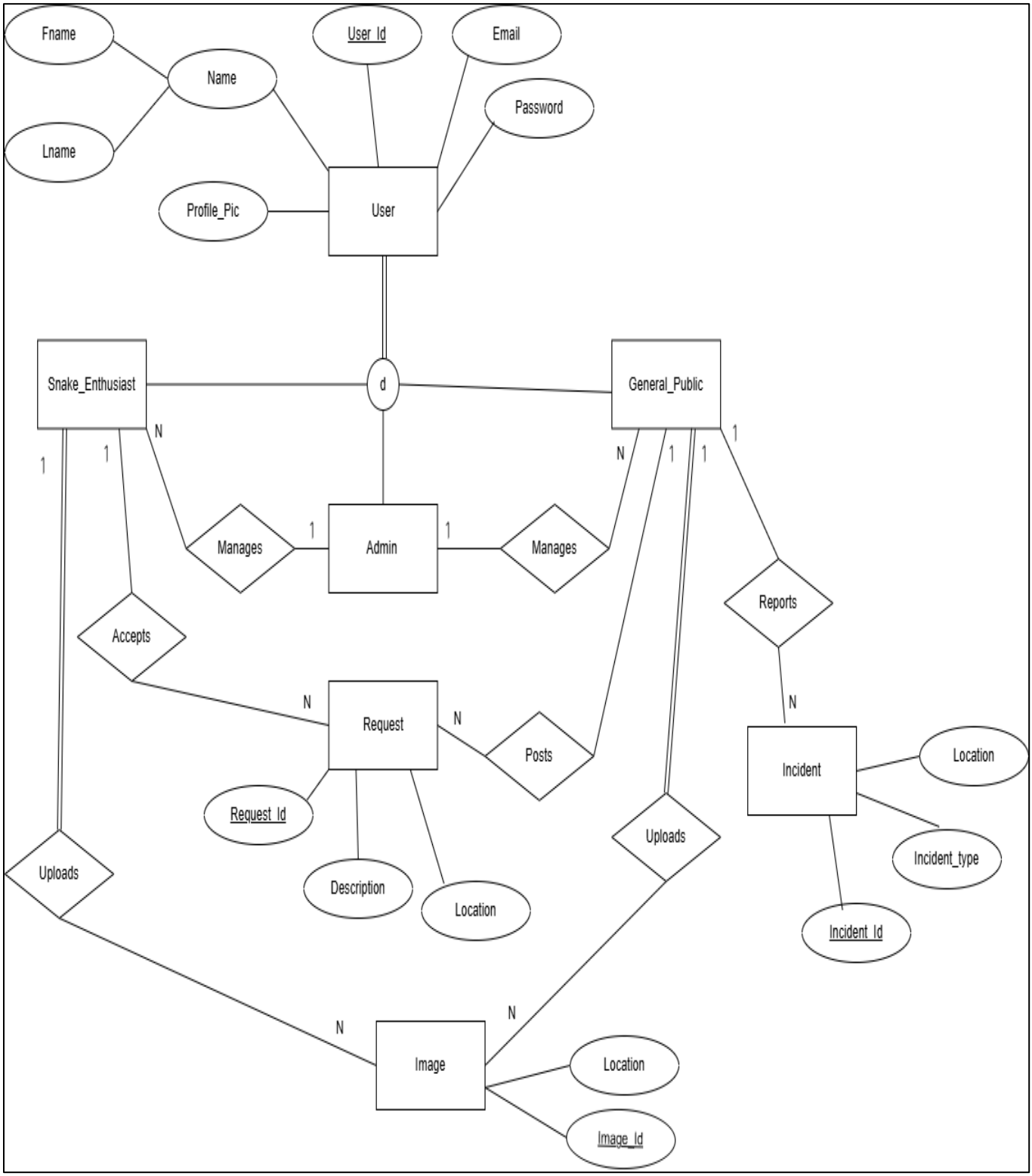


Figure 15 : EER Diagram

4.4 Logical database design

When considering the logical database design, there are 7 steps.

4.4.1 Map Regular Entities

- **User Strong Entity**

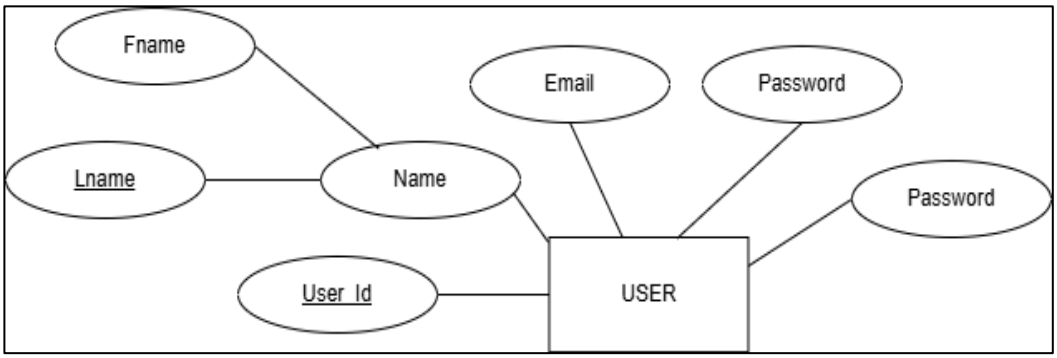


Figure 16 : User Strong Entity

USER (User_Id , Email, Fname, Lname, Password)
Primary key : User Id

- **Name Composite Attribute**

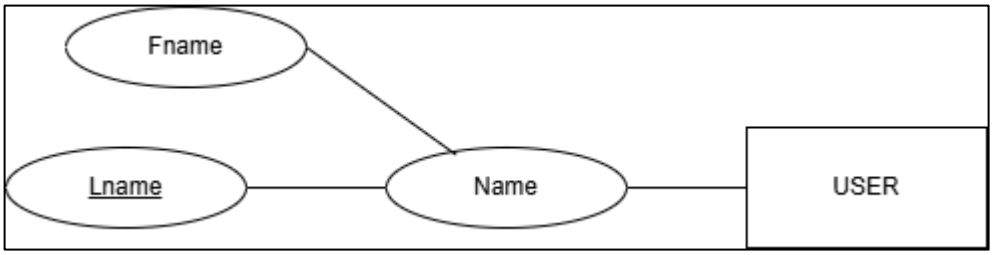


Figure 17 : Name Composite Attribute

USER (User_Id, Fname, Lname)
Primary key : User_Id

- **General public Strong Entity**

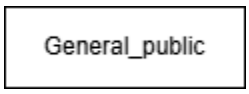


Figure 18 : General Public strong entity

GENERAL_PUBLIC (User_Id)
Primary key : User_Id

- **Snake Enthusiast Strong Entity**

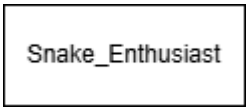


Figure 19 : Snake Enthusiast Strong Entity

SNAKE_ENTHUSIAST (User_Id)
Primary key :User_Id

- **Image Strong Entity**

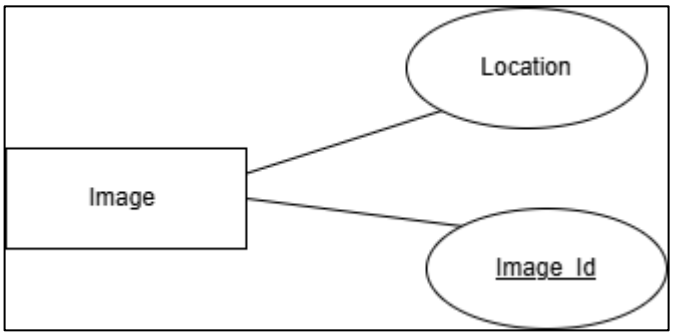


Figure 20 : Image Strong Entity

IMAGE (Image_Id, Location)
Primary key : Image_Id

- **Incident Strong Entity**

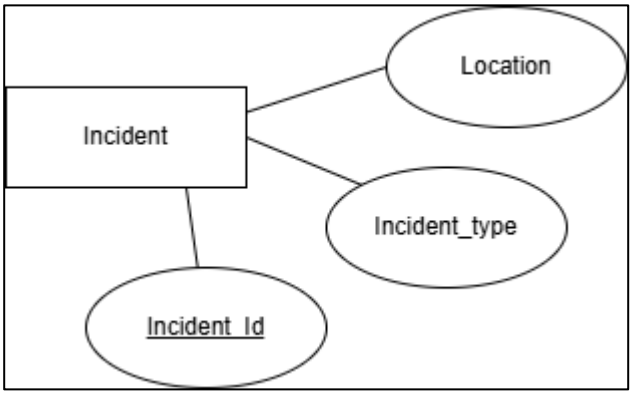


Figure 21 : Incident Strong Entity

INCIDENT (Incident_id, Incident_type,Location)
Primary key : Incident_Id

- **Request Strong Entity**

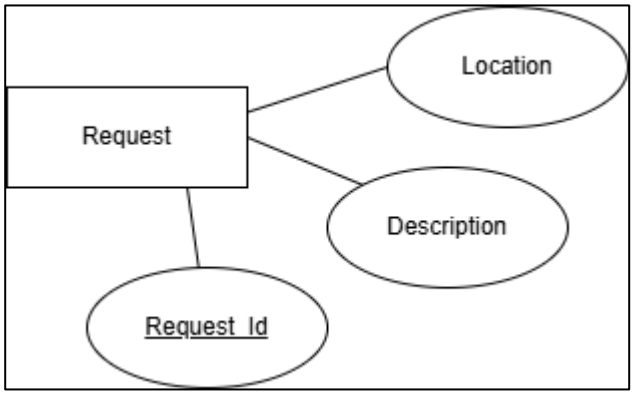


Figure 22 : Request Strong Entity

REQUEST (Request_Id, Description,Location)
Primary key : Request_Id

4.4.2 Map weak Entities

There is no any map associated associated relationships in EER diagram. Therefore, this step has been skipped.

4.4.3 Binary Relationships

Our EER diagram include one to many (1: N) relationships and one to one relationship (1:1). But it is not included many to many (M: N) relationships.

- **Map binary one to many relationships**

  - Consider only General public and Image Entities

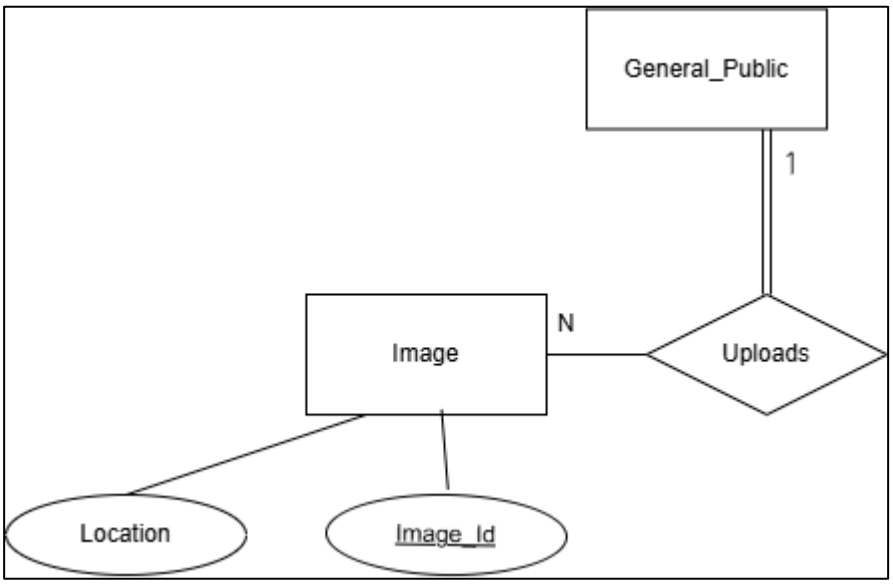


Figure 23 : General public and Image relationship

GENERAL_PUBLIC (User_Id)
Primary key : User_Id
IMAGE (Image_Id, Location)
Primary key : Image_Id

- Consider only General Public and Request Entities

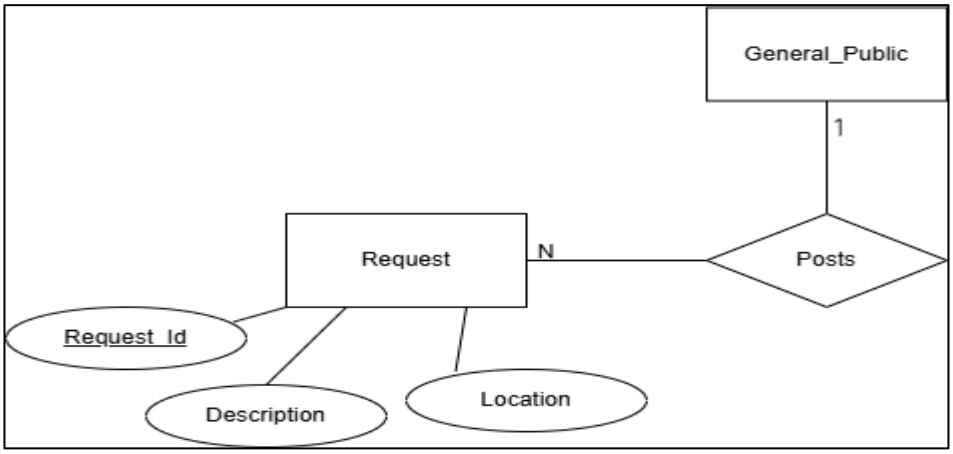


Figure 24 : General public and request relationship

GENERAL_PUBLIC (User_Id)
Primary key : User_Id
REQUEST (Request_Id,Description,Location)
Primary key : Request_Id

- Consider only General public and Incident Entities

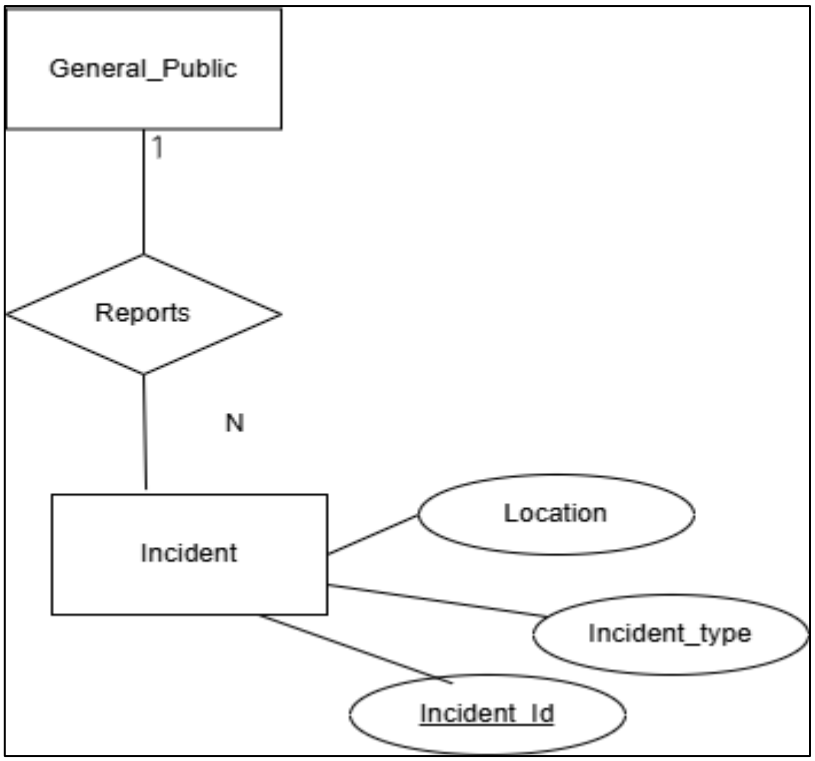


Figure 25 : General Public and Incident relationship

GENERAL_PUBLIC (user_id)
Primary key : User_Id
INCIDENT (Incident_Id, Incident_type, Location)
Primary key : Incident_Id

- Consider only Snake enthusiast and Request Entities

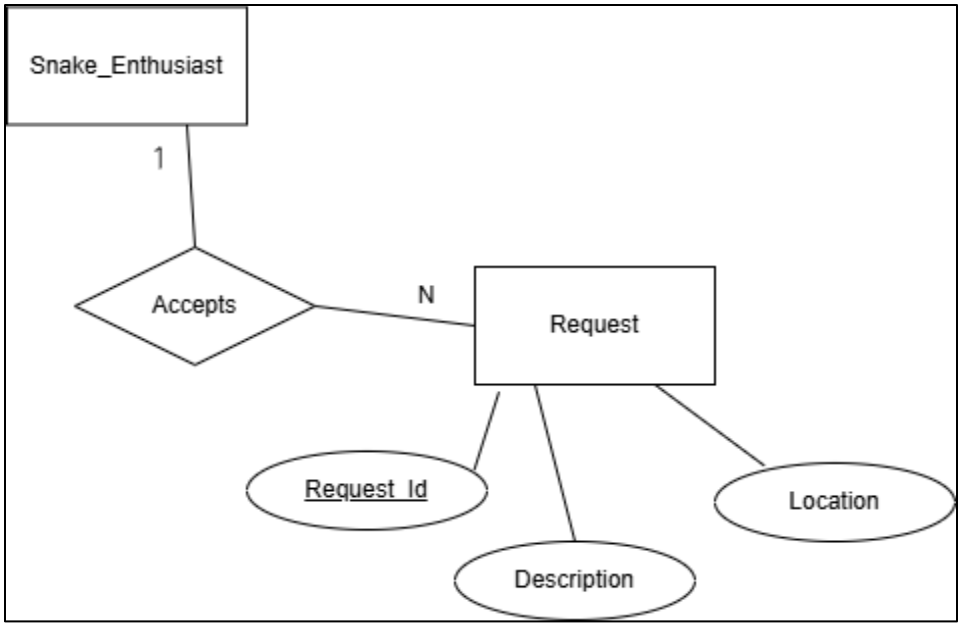


Figure 26 : Snake enthusiast and request relationship

SNAKE_ENTHUSIAST (User_Id)
Primary key : User_Id
REQUEST (Request_id, Description, Location)
Primary key : Request_Id

- **Map binary one to one relationships**

There is no any binary one to one relationships in EER diagram.

4.4.4 Unary Relationships

4.4.5 Map ternary (and n-ary) Relationships

- There is no any ternary (and n-ary) relationhips in EER diagram.

4.4.6 Map super type/subtype relationships

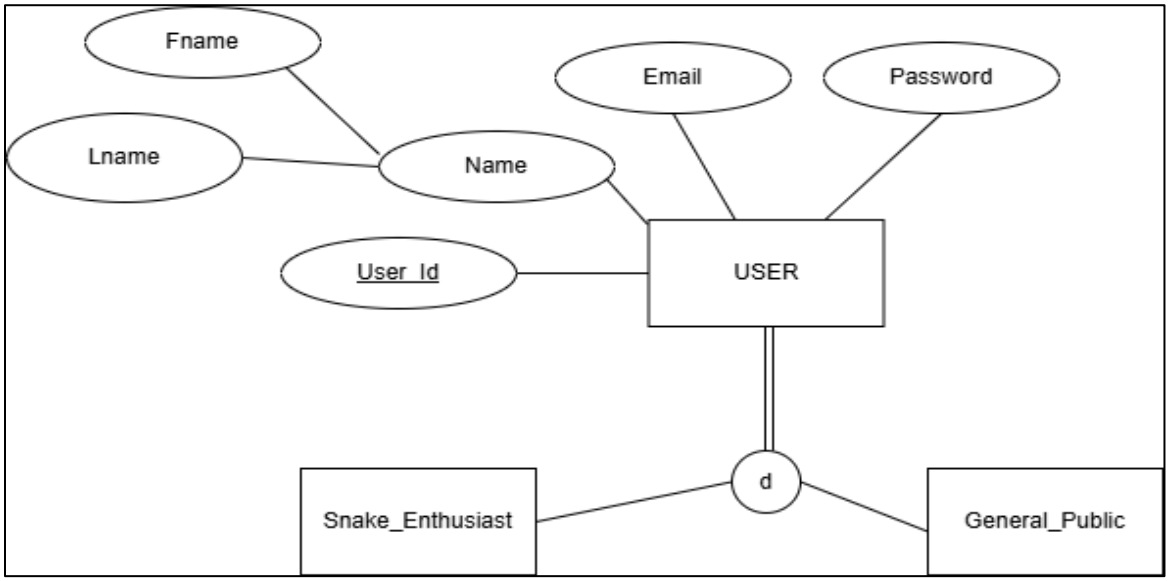
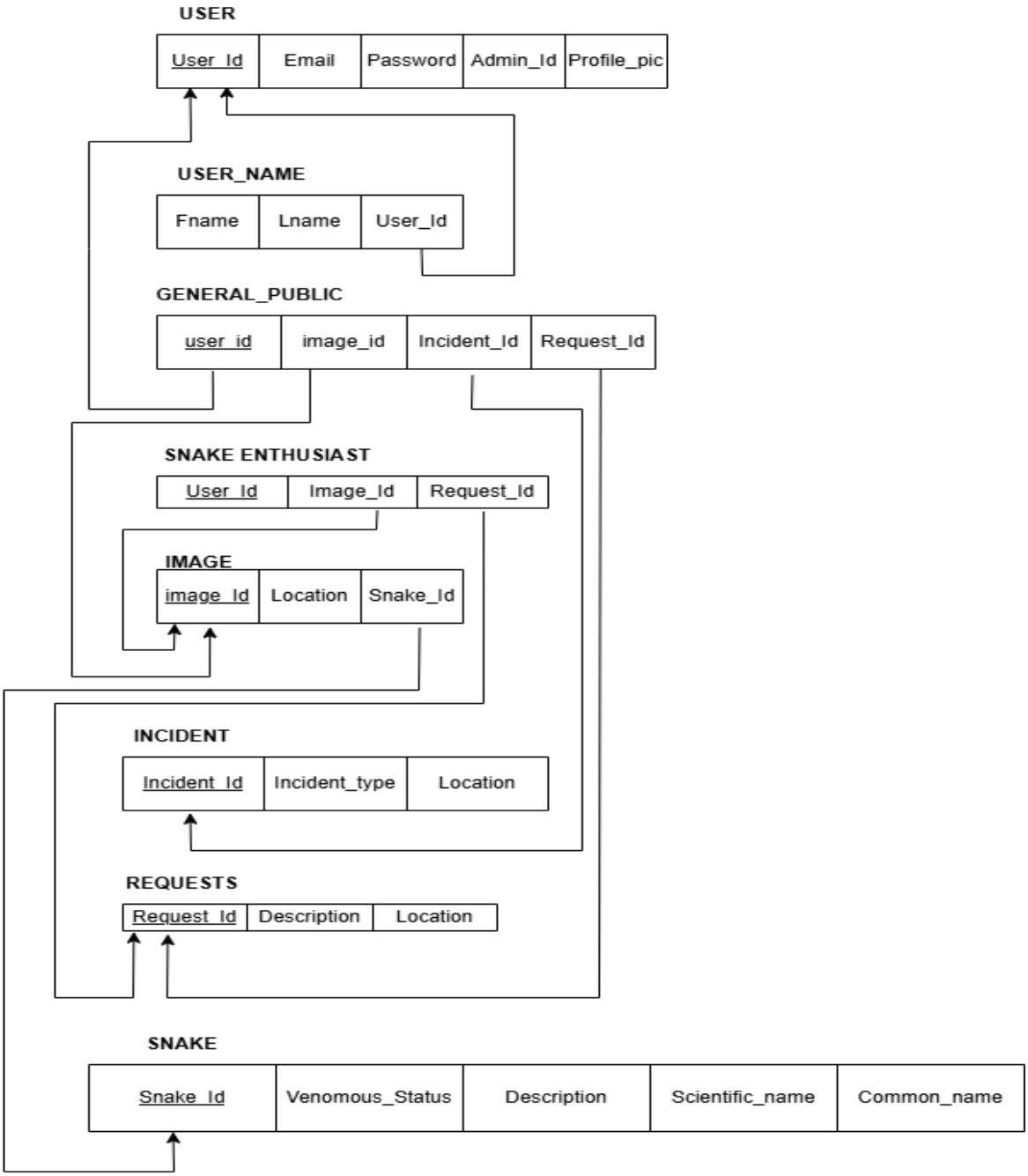


Figure 27 :Supertype / subtype relationship

4.4.7 Map Aggregation

- There are no any aggregations in EER diagram.

4.5 Schema refinement



4.6 Physical database design

4.6.1 Data type Using refine tables

4.7 Security design

Database security design is the strategic process of planning and implementing layers of protection around a database to ensure its Confidentiality, Integrity, and Availability. It is not a single setting, but a comprehensive architecture that begins during the design phase. It includes creating rules and protocols to make sure that only the right people can access the data and that it stays intact and available when needed. This might involve things like setting up privileges, passwords, encryption, and regular checks to make sure everything's working as it should. The aim is to keep sensitive information safe from hackers and other threats, making sure your database is reliable and compliant with rules and regulations.

The security features that are planned to be implemented in our database are,

CHAPTER 05: RESEARCH DESIGN

Snakebite is a major public health issue in many tropical and rural regions, especially in countries like Sri Lanka. Many snakebite incidents occur due to fear, panic, and the inability of people to correctly identify snake species. Misidentification often leads to unnecessary killing of harmless snakes or delays in seeking proper medical treatment. Therefore, there is a strong need for a technological solution that can assist people in identifying snakes quickly and accurately.

With the widespread use of smartphones and recent advancements in artificial intelligence and machine learning, mobile applications have become powerful tools for solving real-world problems. Image recognition technology, particularly deep learning, has shown high accuracy in object classification tasks. This research proposes the design and development of a mobile application for real-time snake identification using machine learning techniques.

This Research Design document explains the overall plan, structure, methods, tools, and procedures used to carry out the research. It provides a clear roadmap of how the research objectives are achieved, from data collection to system evaluation.

5.1 Study objectives-based literature review

The literature review for this research is conducted by aligning existing studies with the core objectives of developing a mobile application for real-time snake identification using Convolutional Neural Networks (CNNs). Previous research in the domain of computer vision highlights CNNs as one of the most effective deep learning techniques for image-based classification tasks. CNNs have been successfully applied in areas such as wildlife monitoring, medical imaging, and object recognition due to their ability to automatically extract hierarchical features from raw image data.

Several studies demonstrate the effectiveness of CNN architectures such as VGGNet, ResNet, and MobileNet in classifying biological species. Among these, MobileNet has gained significant attention for mobile and embedded applications because of its lightweight architecture and reduced computational complexity. Research also emphasizes the importance of preprocessing techniques, such as normalization and image augmentation, to enhance model accuracy under varying lighting and environmental conditions. However, despite these advancements, limited research focuses specifically on real-time snake identification in mobile environments, particularly in regions like Sri Lanka where snake-human encounters are frequent. This study builds upon existing literature by proposing a real-time, mobile-friendly CNN-based solution aimed at improving public safety and awareness.

5.2 Formalizing high-level implementation components

The proposed system is designed using a high-level modular architecture to ensure efficiency, scalability, and maintainability. The system consists of four primary components: the mobile application interface, image preprocessing module, CNN-based snake identification model, and the result visualization and information module. Each component plays a distinct role while maintaining seamless communication with other modules.

The mobile application functions as the primary interaction point for users, allowing them to capture or upload images during a snake encounter. Once an image is provided, it is forwarded to the preprocessing module, which prepares the image by resizing, normalizing, and correcting distortions caused by camera quality or lighting conditions. The processed image is then passed to the CNN model, which performs feature extraction and classification. The final output, including snake species details, venom status, and confidence level, is presented to the user through a clear and intuitive interface. This component-based design ensures real-time responsiveness and future extensibility of the system.

5.3 Data extractions, sample design, test data sets, training data sets

The dataset used in this research consists of labeled images of snake species collected from publicly available datasets, research repositories, and verified online sources. Data extraction involves selecting images that accurately represent snake species under different environmental conditions. Images with poor quality, incorrect labels, or excessive noise are excluded to ensure dataset reliability.

The selected dataset is divided into training, validation, and testing data sets. The training dataset is used to train the CNN model and contains the majority of the images. The validation dataset is used during the training phase to monitor model performance and adjust hyperparameters, helping to prevent overfitting. The test dataset is kept separate and used only for final performance evaluation. To enhance model robustness and generalization, data augmentation techniques such as rotation, flipping, zooming, and brightness adjustment are applied. These techniques help the model adapt to real-world variations encountered during mobile usage.

5.4 Non-functional aspects

The non-functional aspects of the proposed system describe the quality attributes that influence the overall performance, reliability, and efficiency of the mobile application for real-time snake identification. These aspects are critical to ensuring that the system operates effectively in real-world conditions while maintaining accuracy and responsiveness.

CNN Architecture Performance and Reliability

The design of the Convolutional Neural Network (CNN) architecture focuses on achieving high classification accuracy and reliability. System performance is evaluated by comparing predicted snake species with verified ground truth data. Standard evaluation metrics such as accuracy, precision, recall, and F1-score are used to measure the effectiveness of the model. Particular emphasis is placed on the reliable distinction between venomous and non-venomous snake species, as incorrect classification could lead to serious safety risks. The architecture is designed to minimize misclassification while maintaining stable and consistent prediction performance.

Dataset Diversity and Model Robustness

The system design incorporates a diverse and representative dataset to ensure robustness and generalization. The training dataset includes images of Sri Lankan snake species captured under varying environmental conditions, such as different lighting levels, backgrounds, angles, and snake postures. This diversity allows the CNN model to learn discriminative features that remain effective across real-world scenarios. Data augmentation techniques are integrated into the design to reduce overfitting and enhance the model's ability to perform reliably in unseen conditions.

Model Efficiency and Optimization for Mobile Deployment

Efficiency and optimization are essential non-functional considerations due to the mobile nature of the application. The CNN model is designed to be computationally lightweight to ensure fast inference and low latency on mobile devices. Transfer learning techniques using optimized architectures such as MobileNet are adopted to balance accuracy and processing speed. The design minimizes computational overhead to reduce battery consumption and avoid excessive processing delays, ensuring smooth real-time performance without compromising user experience.

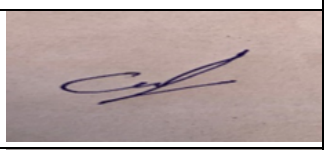
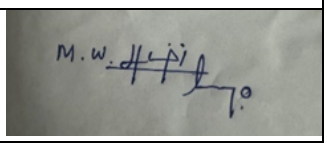
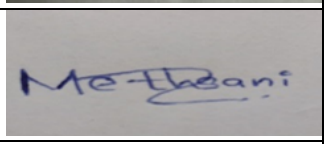
5.5 Proposed validation methods and measurements

The proposed system is validated using both quantitative and qualitative evaluation methods. Quantitative validation includes standard classification performance metrics such as accuracy, precision, recall, and F1-score. These metrics provide a comprehensive assessment of the CNN model's effectiveness in identifying snake species. A confusion matrix is used to analyze correct and incorrect predictions and to identify potential misclassification trends.

In addition to classification accuracy, system response time is measured to evaluate real-time performance on mobile devices. The model is tested using unseen images from the test dataset to simulate real-world conditions. Qualitative validation involves user-based testing to assess ease of use, clarity of results, and overall system satisfaction. The combination of these validation methods ensures that the proposed system is both technically sound and practically useful

CHAPTER 06: APPROVAL

Signature of the team members:

Registration Number	Index Number	Name	Signature
ICT/2021/100	5256	M.A.N.S.G. Samarajeewa	
ICT/2021/032	5193	M.E.T. Fernando	
ASP/2021/045	5387	M.W. Hansaji Lalithya	
ICT/2021/087	5243	M.M.M. Peiris	
ICT/2021/110	5265	H.M.R.A. Senevirathna	

Supervisor's Approval:

I agree / disagree with the design aspects stipulated in this Software Design Specification

Supervisor Name: …….

Department/ Organization: …….

Signature: ……

References

[1] L. J. M. Wickramasinghe, “from Harmless Snakes of Sri Lanka”.

[2] D. D. K. R. W. Dandeniya, B. C. T. Wickramasinghe, and C. Dasanayaka, “A Web-based Application for Snake Species Identification using Vision Transformer and CNN-based Ensemble Meta Classifier,” in *2022 IEEE Pune Section International Conference (PuneCon)*, Pune, India: IEEE, Dec. 2022, pp. 1–5. doi: 10.1109/PuneCon55413.2022.10014812.

[3] S. Abayaratne, W. Ilmini, and T. Fernando, “Identification of Snake Species in Sri Lanka Using Convolutional Neural Networks”.

[4] D. S. Ediriweera *et al.*, “Mapping the Risk of Snakebite in Sri Lanka - A National Survey with Geospatial Analysis,” *PLoS Negl. Trop. Dis.*, vol. 10, no. 7, p. e0004813, Jul. 2016, doi: 10.1371/journal.pntd.0004813.

[5] S. Shahmy, S. A. M. Kularatne, I. B. Gawarammana, S. S. Rathnayake, and A. H. Dawson, “Compliance with national snakebite treatment guidelines in rural Sri Lankan hospitals: a cluster randomized controlled trial of a brief educational intervention,” *BMC Med. Educ.*, vol. 23, no. 1, p. 390, May 2023, doi: 10.1186/s12909-023-04375-1.

[6] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA: IEEE, Jun. 2016, pp. 779–788. doi: 10.1109/CVPR.2016.91.

[7] K. Shivaraj and A. Lakshmi, “Real Time Snake Detection and Alert System,” *Int. J. Eng. Res.*, vol. 13, no. 5, 2024.

[8] Z. Yang and R. Sinnott, “Snake Detection and Classification using Deep Learning,” in *Proceedings of the Annual Hawaii International Conference on System Sciences*, Hawaii International Conference on System Sciences, 2021. doi: 10.24251/hicss.2021.148.

[9] M. Iguernane, M. Ouzziki, Y. Es-Saady, M. El Hajji, A. Lansari, and A. Bouazza, “Deep Learning-Based Snake Species Identification for Enhanced Snakebite Management,” *AI*, vol. 6, no. 2, p. 21, Jan. 2025, doi: 10.3390/ai6020021.

[10] M. S. G. Al-jumaili and M. Shkoukani, “Classification of Venomous and Non- venomous Snakes Using Machine Learning”.

[11] K. Ahmed, M. A. Gad, and A. E. Aboutabl, “Snake species classification using deep learning techniques,” *Multimed. Tools Appl.*, vol. 83, no. 12, pp. 35117–35158, Sep. 2023, doi: 10.1007/s11042-023-16773-0.

[12] N. Parida, A. Narendra, P. R. Kolimi, P. Panda, and I. Misra, “Deep Learning Based Classification of the Big Four Snake Species Using Visual Features,” in *2024 14th International Conference on Cloud Computing, Data Science & Engineering (Confluence)*, Noida, India: IEEE, Jan. 2024, pp. 893–896. doi: 10.1109/confluence60223.2024.10463276.

[13] P. D. R. Deshan, D. V. H. Pabasara, N. A. Yapa, D. S. R. C. V. Perera, D. Lunugalage, and J. L. Wijekoon, “Smart Snake Identification System using Video Processing,” in *TENCON 2021 - 2021 IEEE Region 10 Conference (TENCON)*, Auckland, New Zealand: IEEE, Dec. 2021, pp. 539–544. doi: 10.1109/tencon54134.2021.9707360.

[14] M. Vasmatkar, I. Zare, P. Kumbla, S. Pimpalkar, and A. Sharma, "Snake Species Identification and Recognition," in *2020 IEEE Bombay Section Signature Conference (IBSSC)*, Mumbai, India: IEEE, Dec. 2020, pp. 1–5. doi: 10.1109/ibssc51096.2020.9332218.
[15] Z. Othman *et al.*, "Snake Species Identification using Digital Image Processing," in *2021 6th IEEE International Conference on Recent Advances and Innovations in Engineering (ICRAIE)*, Kedah, Malaysia: IEEE, Dec. 2021, pp. 1–6. doi: 10.1109/icraie52900.2021.9703898.